



Amazon Rekognition Image Analysis Pipeline

Automated Identity Verification for Ghana Card KYC

David Osei Kumi · Edmund Amah · Emmanuel Dako-Mamphey · Nana Kwaku Baffour-Gyawu · Michael Owusu Annor

CLOUD ENGINEERING

APRIL 2026

THE PROBLEM

Manual Identity Verification Does Not Scale

Ghana's Communications Minister mandated SIM re-registration through the National Communications Authority — millions of mobile subscribers must verify their identity or lose service access

The existing process relies on manual agents at physical service centres verifying IDs by eye, creating long queues and operational bottlenecks

No automated face comparison means someone can walk into a centre with another person's Ghana Card and register a SIM in their name

No audit trail exists — there is no tamper-proof record of who was verified, when, by whom, or with what level of confidence



Operational bottleneck

Physical counters cannot handle thousands of concurrent verification requests



Cost at scale

Staffing, rent, and infrastructure overhead for every service centre nationwide



Identity fraud

No face matching means spoofing is undetectable — the exact vulnerability this mandate was designed to close

OUR SOLUTION

Automated Cloud-Based KYC Verification

The Ghana Card already carries everything needed for verification; photo, name, and a unique PIN issued by the National Identification Authority. We built a serverless pipeline on AWS that takes a Ghana Card photo and a live selfie, extracts identity data via OCR, compares the faces, and returns a PASS/FAIL verdict in under 4 seconds. No agents. No queues. Scales to thousands of concurrent requests.

<3s

Verification time

98%+

Face match accuracy

\$0.001

Cost per verification

100%

Audit trail coverage

System Design

Amazon Rekognition	Core AI/ML service. DetectLabels, DetectFaces, DetectText, and CompareFaces APIs
AWS Lambda	Two serverless functions: image analysis handler and KYC verification handler (Python 3.12)
Amazon S3	Four buckets: image inputs, analysis outputs, KYC sessions, and frontend static hosting
Amazon DynamoDB	Stores every KYC verification result with session ID, timestamp, status, and extracted data
API Gateway	HTTP API exposing POST /kyc endpoint with CORS for the frontend
AWS IAM	Least-privilege execution role with scoped permissions for Rekognition, S3, and DynamoDB
Amazon CloudWatch	Log groups with 14-day retention and error alarms for both Lambda functions
Terraform	Infrastructure-as-Code provisioning all AWS resources in a single repeatable script
GitHub Actions	CI/CD pipeline automating Lambda deployment and frontend upload on every push

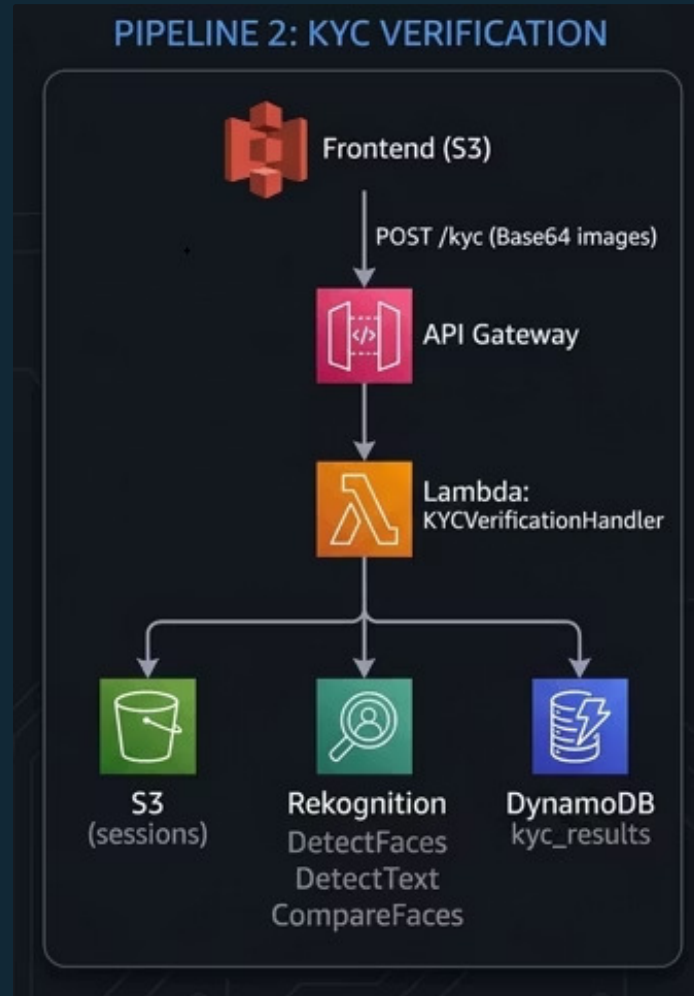
Pipeline 1: General Image Analysis Architecture

Simplified flow from S3 image inputs through Lambda handler to Rekognition for analysis and S3 for outputs



Pipeline 2: KYC Verification Architecture

End-to-end flow from Frontend (S3) through API Gateway, Lambda handler, to Rekognition and DynamoDB for session management and face detection



How It Works — KYC Verification Flow

O1

User uploads Ghana Card and selfie

Images are Base64 encoded in the browser and sent to API Gateway via POST /kyc

O2

Lambda stores images in S3 and validates faces

Both images are uploaded to the KYC sessions bucket. DetectFaces confirms a face exists in each

O3

DetectText extracts name and ID number from the card

OCR reads all text on the card. The system identifies the surname, forenames, and Personal ID Number using positional and label-based extraction

O4

CompareFaces matches the ID photo to the selfie

Rekognition compares the two faces with a 70% similarity threshold. Above 70% is a PASS

O5

Result stored in DynamoDB, returned to frontend

PASS or FAIL verdict with face confidence, extracted name, ID number, and session ID. Full audit trail

Terraform — Infrastructure as Code

All AWS resources are defined in Terraform configuration files. A single terraform apply provisions the complete infrastructure from scratch. Every resource is repeatable, version-controlled, and destroyable.

lambda.tf
Two Lambda functions with Python 3.12, 256MB memory, 60s timeout, and automatic code packaging from source

s3.tf
Four S3 buckets with event notifications, CORS policies, and static website hosting for the frontend

api_gateway.tf
HTTP API with POST /kyc route, Lambda integration, CORS enabled for all origins

dynamodb.tf
KYC results table with PAY_PER_REQUEST billing and session_id as the primary key

iam.tf
Least-privilege execution role with scoped access to Rekognition, S3, DynamoDB, and CloudWatch

cloudwatch.tf
Log groups with 14-day retention and error alarms that trigger when either Lambda throws an exception

Error Handling

Unsupported file format	Returns clean message: "Use JPEG or PNG"
File exceeds 5MB	Rejected before Rekognition is called
No face in ID card	Returns specific guidance to user
No face in selfie	Returns specific guidance to user
Rekognition throttled	Automatic retry (3 attempts, 2s delay)
Unhandled exception	Logged to CloudWatch with full context

CloudWatch Monitoring

Both Lambda functions log to dedicated log groups with 14-day retention. Every request, every result, every error is recorded.

Two alarms monitor Lambda error rates. Any unhandled exception triggers the alarm within 60 seconds for immediate investigation.

CI/CD Pipeline — GitHub Actions

Every push to the main branch triggers an automated deployment. No manual steps, no forgetting to redeploy, no deploying stale code.



1 Checkout

GitHub pulls the latest code onto a clean Ubuntu runner



2 AWS Authentication

Pipeline authenticates with AWS using secrets stored securely in GitHub



3 Package Lambda

Zips the src/ folder into a fresh deployment package



4 Deploy

Pushes the zip to both Lambda functions and uploads the frontend to S3

Demo Verification Output

PASS Case — Matching Identity

Status	PASS
Face Match Confidence	98.39%
Extracted Name	KWAME ADU MENSAH
Extracted ID Number	GHA0087654321
Verification Time	< 4 seconds

FAIL Case — Mismatched Identity

Status	FAIL
Face Match Confidence	~10-15%
Extracted Name	Correctly extracted from card
Extracted ID Number	Correctly extracted from card
Fraud Detection	Selfie does not match ID photo

📄 Every verification result (PASS and FAIL) is automatically written to the DynamoDB audit table with a unique session ID, timestamp, and all extracted data points.

Bonus Features Implemented



Web Frontend

Responsive single-page application hosted on S3 with static website hosting. Upload Ghana Card and selfie directly from the browser.



DynamoDB Audit Dashboard

Every verification stored with session ID, timestamp, status, confidence, extracted name, and ID number for compliance and audit.



Face Comparison (CompareFaces)

Real identity verification using face matching between the ID card photo and a live selfie with 70% confidence threshold.



CI/CD Pipeline

GitHub Actions automates deployment on every push to main. Lambda code and frontend deployed without any manual steps.

Challenges and How We Solved Them

1

Ghana Card name extraction inaccurate

Root Cause: Rekognition returns field labels and values as separate unlinked lines with no guaranteed structure

Solution: Switched from label-based to positional extraction. First name-like line is surname, second is forenames (matching Ghana card layout)

2

Personal ID Number (GHA-...) not detected

Root Cause: The GHA number is printed on the holographic section of the card, making it difficult for Rekognition to read

Solution: Implemented 5-strategy fallback: word-level GHA scan, line-level scan, label search at multiple confidence thresholds, Document Number as last resort

3

Unsupported image format crashes Lambda

Root Cause: `InvalidImageFormatException` from Rekognition returned raw AWS error to user

Solution: Added specific exception catch for `InvalidImageFormatException` returning a clean user-friendly message

4

CI/CD pipeline failing with "already exists" errors

Root Cause: Terraform state was local, so the pipeline tried to recreate all infrastructure from scratch

Solution: Separated concerns: infrastructure managed locally via Terraform, pipeline deploys code only

Future Enhancements

Identity Verification

→ Liveness Detection

Rekognition Face Liveness API to close the photo-spoofing vulnerability and prevent identity fraud with printed photos or screens

→ NIA Database Integration

Validate extracted data directly against the Ghana National Identification Authority API for real-time confirmation

→ Open-Source Release

Publish the full pipeline on GitHub so other developers and institutions can build on it

Beyond KYC — Rekognition at Scale

→ Disease Detection

Extend the image analysis pipeline to detect plant and crop diseases from uploaded field photos

→ Agricultural Analysis

Use Rekognition to classify crop health, pest damage, and yield estimates for farming communities

→ Batch Processing

Process entire folders of images uploaded to S3 and generate aggregated summary reports

Thank You

Questions?

Repository: github.com/dkumi12/project-rekognition

Frontend: rekkyk-frontend.s3-website-us-east-1.amazonaws.com